

Stage 2 Report:

Predicting Helsinki City Bike Trip Duration from Route, Time and Weather

CS-C3240 Machine Learning

1. Introduction

Shared city bikes are popular in Helsinki and Espoo, but riders cannot easily tell in advance how long a trip will take. A good estimate would help people decide whether to cycle, for example in bad weather or at a busy time of day. This project predicts how long a city bike trip will take before it starts. Section 2 defines the problem and the data, Section 3 describes our methods, Section 4 presents the results, and Section 5 summarises what we learned.

2. Problem Formulation

This is a supervised regression problem. Each data point is one completed bike trip, and the label is the trip's duration in minutes. Each trip is described by seven features (Table 1): its route, when it starts, and the weather that day.

Table 1. Features describing each trip.

Feature	Type	Description
Route length	Continuous	Typical distance (m) between the start and end station, from past trips
Round trip	Binary	Whether the bike is returned to the station it was taken from
Weekday	Categorical	Day the trip started (Mon-Sun)
Hour of day	Categorical	Hour the trip started (0-23)
Month	Categorical	Month of the season (Apr-Oct)
Temperature	Continuous	Daily mean air temperature (°C)
Precipitation	Continuous	Daily total precipitation (mm)

The problem is the same as in Stage 1, but we added the route. In Stage 1 we left out distance entirely, so that the model could not see the answer. On reflection, that mixed up two different things. The distance a trip actually covered is only known after the ride, so we still leave it out. But riders know where they are going before they start, so the typical length of their route is fair to use.

The trip data comes from HSL's open dataset of city bike trips [1]. The weather data comes from the Finnish Meteorological Institute's open data for the Kaisaniemi station in Helsinki [2], and is matched to each trip by date.

3. Methods

3.1 Data and preprocessing

We use all trips from the April-October seasons of 2022-2025, about 10 million in total. We removed incomplete rows, "false starts" shorter than 10 seconds or 10 metres (a bike taken out and put straight back), and bikes moved to HSL's workshop or test stations. This leaves **9,837,352 trips**. We kept very long trips, most of which are bikes that were never properly returned, because there is no natural cut-off for removing them. Section 3.4 explains how we handle them instead.

3.2 Feature selection

We started from three assumptions: longer trips take longer, the time of day affects speed (through traffic or the kind of trip), and bad weather slows riders down. Since duration is distance divided by speed, we plotted the typical speed and the typical distance against each factor. Distance varied far more than speed, so trip length is the main thing to capture. Speed changed somewhat with the time of day: riders were fastest in the morning, when many of them are commuting, and slower in the afternoon. This suggests that the purpose of the trip matters more than traffic. Rain did not slow riders down, contrary to our Stage 1 assumption; on rainy days, people simply made shorter trips. We kept the Stage 1 time and weather features so that we could test their effect, and added two route features.

Route length is the median distance covered by other trips between the same two stations. To avoid leaking information, it is only computed from other seasons: 2022 training trips use 2023 trips and vice versa, validation trips use 2022-2023, and test trips use 2022-2024. Routes with fewer than three past trips use the median for their starting station instead. The **round-trip** flag marks trips that end where they started, which are mostly leisure rides.

3.3 Models

We compare two hypothesis spaces covered in the course [3]:

- **Linear regression.** We predict the logarithm of duration from the logarithm of route length and the other features. This matches the idea that duration is distance divided by speed, where time and weather make riders a few percent faster or slower.
- **Gradient-boosted regression trees** [5]. Trees can capture interactions that a linear model cannot, for example rain mattering more on long trips. We include them to check whether such interactions exist.

To see how much the models actually learn, we compare them with two simple baselines: always predicting the median duration, and a **physics baseline** that divides route length by one typical speed (11.5 km/h, estimated from the training data).

3.4 Loss functions

A small number of trips last hours or even days, mostly bikes that were never returned. With the usual squared error loss, these few trips would dominate training: trips longer than five hours are only 0.3% of the data, but they make up almost all of the squared error. We handle this in two ways. First, we predict the logarithm of the duration, which shrinks these extreme values. Second, we train the linear model with the **Huber loss** [4] ($\epsilon = 1.35$). It treats small errors like the squared error, but large errors only linearly, so a few extreme trips cannot pull the model too far. To check whether this matters, we also train the same linear model with the **squared error**. For the trees we use the **absolute error**, which is also robust to outliers; the library we used does not offer the Huber loss for trees.

This gives three methods: linear regression with squared error, linear regression with Huber loss, and trees with absolute error. We evaluate them in minutes, using the mean absolute error (MAE), the median absolute error (MedAE, the error on a typical trip) and the **share of trips with an error of at most 2 minutes**: trips whose predicted duration is within 2 minutes of the actual duration, however long the trip is.

3.5 Training, validation and test sets

We split the data by season: 2022-2023 for training (4,876,755 trips), 2024 for validation (2,508,741 trips) and 2025 for testing (2,451,856 trips). This mimics real use, where the model predicts trips in a future season. A random split would put trips from the same days into both training and validation, which would make the results look better than they are. To keep training fast, every model is fitted on the same random sample of 400,000 training trips. We decided in advance to choose the method with the lowest validation MAE, retrain it on 2022-2024, and evaluate it once on the test set. Because the differences between methods are small, we also compute 95% confidence intervals for them by resampling whole days [6].

4. Results

4.1 Comparing the methods

Table 2. Errors in minutes, and share of trips with an error of at most 2 minutes. The chosen method is in bold.

Method	Training		Validation (2024)			Test (2025)		
	MAE	MedAE	MAE	MedAE	Error ≤ 2 min	MAE	MedAE	Error ≤ 2 min
Constant (median)	10.40	4.88	13.98	5.30	19.0%	14.89	5.43	18.6%
Physics baseline	7.92	1.76	11.22	2.06	49.1%	11.95	2.14	48.0%
Linear, squared error	7.74	1.98	11.00	2.21	46.1%	11.70	2.35	43.3%
Linear, Huber loss	7.62	1.62	10.91	1.86	52.5%	11.55	1.91	51.5%
Trees, absolute error	7.57	1.58	10.96	1.85	52.5%	11.60	1.89	51.8%

Table 2 shows four things:

- **Route length does most of the work.** The physics baseline only divides route length by a typical speed, yet it already cuts the typical validation error from 5.3 to 2.1 minutes.
- **Machine learning adds a small but real improvement.** The best methods bring the typical error down to about 1.85 minutes and give 3.3 percentage points more trips with an error of at most 2 minutes than the physics baseline (95% CI [3.1, 3.5]). Part of the gain comes from details the simple rule misses: longer routes are ridden faster, so doubling a route makes a trip about 80% longer rather than twice as long.
- **The Huber loss matters.** With squared error, the few extreme trips still pull predictions up by about half a minute, so this method does worse than the physics baseline on typical trips. With the Huber loss, 6.4 percentage points more trips have an error of at most 2 minutes (CI [6.1, 6.6]). Removing the longest 0.1% of training trips also moves the squared-error model's predictions about ten times more than the Huber model's.
- **Trees do not help.** The trees and the Huber model are practically tied. The linear model's MAE is 0.05 minutes (about three seconds) lower, and both have the same share of trips with an error of at most 2 minutes.

Training errors are lower than validation errors for every method, including the constant baseline, which learns almost nothing. The gap therefore comes from 2024 being a harder season, not from overfitting. We choose **linear regression with the Huber loss**: it has the lowest validation MAE, as our rule requires, and it is simpler and easier to interpret than the trees.

4.2 Which features matter

To test our three assumptions, we refitted the chosen model with different sets of features (Table 3).

Table 3. Validation results of the chosen model with different features. "Route" means route length and the round-trip flag.

Features used by the chosen model	Validation MedAE (min)	Error $\leq$ 2 min
None (constant prediction)	5.300	19.0%
Time + weather (the Stage 1 features)	5.250	19.1%
Route	1.865	52.3%
Route + time	1.850	52.6%
Route + weather	1.875	52.2%
Route + time + weather (chosen model)	1.857	52.5%

Route length explains almost everything. Without it, the Stage 1 features barely beat a constant prediction; with route length alone, the model is almost as good as the full model. Time adds a little: on the same route, a trip takes about 10% less time at 5-6 am and about 4% more in the early afternoon than on average, and weekend trips take about 3% longer. Weather adds nothing, and adding it to route length even makes the typical error slightly worse. In the fitted model, 10 mm of rain makes a trip only about 1% shorter.

4.3 Test error

The 2025 test season was not used for training or for choosing the method. On it, the chosen method has a **test MAE of 11.55 minutes and a test MedAE of 1.91 minutes**, and predicts 51.5% of trips to within 2 minutes of their actual duration (Table 2). This is close to its validation results, so the model also works on a new season, and the methods rank the same way as on the validation data. The predictions are not biased: on a typical trip, the ratio of actual to predicted duration is 1.006.

5. Conclusions

We predicted city bike trip durations from the route, the time and the weather. The chosen method, linear regression with the Huber loss, has a typical error of 1.9 minutes on an unseen season, against 5.4 minutes for a constant guess. The main lesson is that route length explains most of the prediction: a simple distance-divided-by-speed rule already does well, and machine learning adds a small but real improvement on top. Training with the Huber loss matters because of the extreme trips, while more complex trees and the weather add little.

For a typical trip, the problem is solved well. The average error (about 11.5 minutes) stays much larger than the typical error because it is dominated by bikes that were never returned and by riders who stop along the way, which our features cannot predict. The results could be improved with hourly instead of daily weather, station coordinates to estimate the length of routes without history, training on all trips instead of a sample, and a separate model that detects bikes that were not returned.

Use of AI

We used generative AI (Claude, by Anthropic) throughout Stage 2. It helped us plan and carry out the analyses, write and debug the notebook code, think through the problem formulation (including the difference between a trip's recorded distance and its route), and draft and edit this report. We made the main design decisions ourselves: adding route length, comparing three methods, and computing route length from past trips. We checked the code, results and text before submitting.

6. References

- [1] HSL (Helsinki Region Transport). Helsinki and Espoo city bike trip data (CC BY 4.0). https://hri.fi/data/en_GB/dataset/helsingin-ja-espoon-kaupunkipyorilla-ajatut-matkat
- [2] Finnish Meteorological Institute. Open data, weather observations (CC BY 4.0). <https://en.ilmatieteenlaitos.fi/open-data>
- [3] Jung, A. Machine Learning: The Basics. <https://mlbook.cs.aalto.fi>
- [4] Huber, P. J. (1964). Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1), 73-101.
- [5] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- [6] Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall/CRC.

7. Appendix: Code

The full notebook (ML_bike.ipynb), which prepares the data and trains and evaluates all methods, is attached on the following pages.

ML-bike: predicting city bike trip duration

The goal is to predict how long a city bike trip will take before it starts, using the route the rider picked, the time they leave and the weather.

Problem setup

- **Data point:** one bike trip.
- **Label:** trip duration in minutes. The models are trained on $\log(\text{duration})$, so each feature changes the duration by a percentage instead of a fixed number of minutes.
- **Features:** route length (the typical distance between the start and end station, learned from earlier trips), a round-trip flag, hour of day, weekday, month, daily mean temperature and daily precipitation.
- **Models:** linear regression and gradient-boosted regression trees.
- **Loss:** squared error and Huber loss for linear regression, absolute error for the trees.
- **Split:** by season. Train on 2022–2023, validate on 2024, and test once on 2025.

Change since Stage 1

In Stage 1 we left distance out so the model couldn't "cheat". That mixed up two things. The distance a trip actually covered is only known after the ride, so it stays out. But the route (which two stations) is known before the trip starts, and our problem already assumes the rider knows where they are going. Without the route, time and weather barely beat a constant prediction (see step 11), so in Stage 2 we add the route length, computed only from earlier trips.

We also cut the comparison down to three models, chosen so that each pair differs in only one thing (step 8).

Data

- `data/YYYY-MM.csv` : HSL city bike trips in Helsinki and Espoo, April–October 2022–2025.
- `data/weather_daily.csv` : daily weather from the Finnish Meteorological Institute (FMI), Helsinki Kaisaniemi station. Step 3 downloads it again if it's missing.

Steps

1. Setup
2. Load the trips
3. Load the weather
4. Merge and clean
5. Very long trips
6. What affects trip duration?
7. Features and data split
8. Train and compare the models
9. Are the differences real?

10. Does the loss still matter?
11. Which features matter?
12. What the model learned
13. Test on the 2025 season
14. Conclusions

1. Setup

In [1]:

```
import ssl
import urllib.parse
import urllib.request
import warnings
import xml.etree.ElementTree as ET
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer, TransformedTargetRegressor
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.linear_model import HuberRegressor, LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

DATA_DIR = Path("data")
WEATHER_CSV = DATA_DIR / "weather_daily.csv"
RANDOM_STATE = 42
TRAIN_SAMPLE = 400_000

pd.set_option("display.float_format", "{:,.3f}".format)
plt.rcParams["figure.figsize"] = (11, 4)
plt.rcParams["axes.grid"] = True
plt.rcParams["grid.alpha"] = 0.3
```

2. Load the trips

All the monthly files have the same columns, and we only load the ones we need: the departure time (for the calendar features and to join the weather), the start and end station (the route), the duration (the label) and the distance the trip covered.

The covered distance is never used as a feature, since it's only known after the trip. We use it to filter out false starts and, from earlier seasons only, to work out how long each route usually is (step 7).

In [2]:

```
COLUMNS = {
    "Departure": "departure",
    "Departure station id": "from_id",
    "Return station id": "to_id",
    "Covered distance (m)": "distance_m",
```

```

        "Duration (sec.)": "duration_s",
    }

files = sorted(DATA_DIR.glob("[0-9]*.csv"))
print(f"Found {len(files)} monthly trip files")

frames = []
for f in files:
    month_df = pd.read_csv(
        f,
        usecols=list(COLUMNS),
        dtype={
            "Departure station id": "str",
            "Return station id": "str",
            "Covered distance (m)": "float32",
            "Duration (sec.)": "float32",
        },
    ).rename(columns=COLUMNS)
    frames.append(month_df)

trips = pd.concat(frames, ignore_index=True)
del frames

# Some rows only have a date and no time. errors="coerce" turns them into NaT; they're dropped in step 4.
trips["departure"] = pd.to_datetime(trips["departure"], format="%Y-%m-%dT%H:%M:%S", errors="coerce")

print(f"Loaded {len(trips):,} trips")
trips.head()

```

Found 28 monthly trip files

Loaded 10,121,026 trips

Out[2]:

	departure	from_id	to_id	distance_m	duration_s
0	2022-04-30 23:59:53	001	004	1,204.000	128,982.000
1	2022-04-30 23:59:39	710	707	559.000	284.000
2	2022-04-30 23:59:19	403	131	2,199.000	1,078.000
3	2022-04-30 23:59:03	039	086	1,933.000	590.000
4	2022-04-30 23:58:55	002	012	1,208.000	477.000

3. Load the weather

Daily observations from FMI's open data API for the Helsinki Kaisaniemi station

(fmisid=100971):

- tday → temp_c : daily mean temperature (°C)
- rrday → precip_mm : daily precipitation (mm)

FMI reports -1 mm on days without measurable rain, so we set those to 0. If the CSV file is missing, the cell below downloads it again, so the notebook can be run from scratch. Hourly

weather would match the trips better, but daily data keeps the join simple.

In [3]:

```
FMISID = 100971 # Helsinki Kaisaniemi station

def download_weather(path: Path, years=range(2022, 2026)) -> None:
    import certifi # without it, HTTPS fails on some Windows Python installs

    context = ssl.create_default_context(cafile=certifi.where())
    ns = "{http://xml.fmi.fi/schema/wfs/2.0}"
    frames = []

    for year in years:
        query = urllib.parse.urlencode({
            "service": "WFS",
            "version": "2.0.0",
            "request": "getFeature",
            "storedquery_id": "fmi::observations::weather::daily::simple",
            "fmisid": FMISID,
            "starttime": f"{year}-04-01T00:00:00Z",
            "endtime": f"{year}-10-31T23:59:59Z",
            "parameters": "tday,rrday",
        })
        with urllib.request.urlopen(f"https://opendata.fmi.fi/wfs?{query}",
            timeout=120, context=context) as r:
            root = ET.fromstring(r.read())

            rows = [
                {
                    "time": el.findtext(f"{ns}Time"),
                    "name": el.findtext(f"{ns}ParameterName"),
                    "value": pd.to_numeric(el.findtext(f"{ns}ParameterValue"),
errors="coerce"),
                }
                for el in root.iter(f"{ns}BswfsElement")
            ]
            wide = pd.DataFrame(rows).pivot_table(index="time", columns="name",
values="value", aggfunc="first")
            wide.index = pd.to_datetime(wide.index).tz_convert(None).normalize()
            frames.append(wide)
            print(f"  downloaded {year}: {len(wide)} days")

    out = pd.concat(frames).sort_index().rename(columns={"tday": "temp_c",
"rrday": "precip_mm"})
    out.index.name = "date"
    out[["temp_c", "precip_mm"]].to_csv(path, float_format="%.1f")

if not WEATHER_CSV.exists():
    print("Downloading weather from FMI open data...")
    download_weather(WEATHER_CSV)

weather = pd.read_csv(WEATHER_CSV, parse_dates=
["date"]).set_index("date").sort_index()
weather["precip_mm"] = weather["precip_mm"].clip(lower=0) # FMI uses -1 for "no
measurable rain"
```

```
print(f"{len(weather)} days, {weather.index.min().date()} -> {weather.index.max().date()}")
print(f"Rainy days (>= 0.1 mm): {(weather['precip_mm'] >= 0.1).mean():.1%}")
weather.describe()
```

856 days, 2022-04-01 -> 2025-10-31

Rainy days (>= 0.1 mm): 39.6%

Out[3]:

	temp_c	precip_mm
count	856.000	856.000
mean	13.035	1.892
std	5.847	4.867
min	-2.000	0.000
25%	8.800	0.000
50%	13.750	0.000
75%	17.625	1.200
max	24.900	43.100

4. Merge and clean

We add the weather to each trip by date and then do some light cleaning:

- remove rows with missing values;
- remove **false starts**: trips under 10 seconds or under 10 metres, where a bike was taken out and put straight back;
- remove trips to or from HSL's **workshop and test stations** (997 , 999). These are maintenance moves, not real rides.

We keep the very long trips, even though most of them are bikes that were never properly returned. There is no obvious place to cut them off, so instead we deal with them through the log transform and the loss function (steps 5 and 8).

At the end, each station gets an integer code and each (start, end) pair becomes a **route**.

In [4]:

```
NON_RIDE_STATIONS = {"997": "Workshop Helsinki", "999": "Relay Box test station / Station Test"}

n_raw = len(trips)

trips = trips.dropna(subset=["departure", "duration_s", "distance_m", "from_id", "to_id"])
trips["from_id"] = trips["from_id"].str.strip()
trips["to_id"] = trips["to_id"].str.strip()

trips["date"] = trips["departure"].dt.normalize()
trips = trips.merge(weather, left_on="date", right_index=True, how="left")
```

```

no_weather = trips["temp_c"].isna()
print(f"Trips with no matching weather day:      {no_weather.sum():,}")
trips = trips[~no_weather]

false_start = (trips["duration_s"] < 10) | (trips["distance_m"] < 10)
non_ride = trips["from_id"].isin(NON_RIDE_STATIONS) |
trips["to_id"].isin(NON_RIDE_STATIONS)
print(f"False starts removed:                  {false_start.sum():,}")
print(f"Workshop / test-station moves removed: {(non_ride &
~false_start).sum():,}")
trips = trips[~false_start & ~non_ride].reset_index(drop=True)

# float64 on purpose: the log transform in step 8 checks that exp(log(y)) gives
# back y,
# and that check fails with float32 precision.
trips["duration_min"] = trips["duration_s"].astype("float64") / 60
trips["distance_km"] = trips["distance_m"].astype("float64") / 1000

# A route is an ordered (start, end) pair. There are fewer than 1000 stations,
# so start * 1000 + end gives every route a unique number.
stations = pd.Index(pd.unique(np.concatenate([trips["from_id"].to_numpy(),
trips["to_id"].to_numpy()])))
trips["from_code"] = stations.get_indexer(trips["from_id"].astype("int32"))
trips["to_code"] = stations.get_indexer(trips["to_id"].astype("int32"))
trips["route"] = trips["from_code"] * 1000 + trips["to_code"]
trips["round_trip"] = (trips["from_code"] == trips["to_code"]).astype("int8")
trips = trips.drop(columns=["from_id", "to_id", "date"])

print(f"\nRaw trips:          {n_raw:,}")
print(f"After cleaning:  {len(trips):,} ({len(trips) / n_raw:.1%}")
print(f"Stations: {len(stations):,} | distinct routes:
{trips['route'].nunique():,} | "
      f"round trips: {trips['round_trip'].mean():.1%}")
trips[["duration_min", "distance_km", "temp_c", "precip_mm"]].describe()

```

```

Trips with no matching weather day:      0
False starts removed:                    274,378
Workshop / test-station moves removed:  1,264
Raw trips:                               10,121,026
After cleaning:  9,837,352 (97.2%)
Stations: 463 | distinct routes: 110,389 | round trips: 3.2%

```

Out[4]:	duration_min	distance_km	temp_c	precip_mm
count	9,837,352.000	9,837,352.000	9,837,352.000	9,837,352.000
mean	18.827	2.549	14.772	1.418
std	250.605	2.685	5.283	4.080
min	0.167	0.010	-2.000	0.000
25%	6.400	1.159	11.000	0.000
50%	10.850	1.986	15.700	0.000
75%	18.283	3.342	18.500	0.600
max	279,454.317	3,681.009	24.900	43.100

5. Very long trips

A few trips last hours or even days, mostly bikes that were never returned. Here we check how much these trips would affect training, which is the reason behind our choice of loss function.

In [5]:

```
d = trips["duration_min"]
q = d.quantile([0.5, 0.9, 0.99, 0.999, 0.9999])

print(f"mean      {d.mean():8.1f} min")
print(f"median    {d.median():8.1f} min")
for level, value in q.items():
    print(f"p{level * 100:<8g} {value:8.1f} min")
print(f"max        {d.max():8.1f} min  ({d.max() / 60:.1f} hours)")
print()
print(f"Trips over 5 hours: {(d > 300).sum():>8,}  ({(d > 300).mean():.3%} of trips)")
print(f"Trips over 24 hours: {(d > 1440).sum():>8,}  ({(d > 1440).mean():.4%} of trips)")

# How much of the total error comes from these few long trips?
tail = d > 300
sq_error = (d - d.mean()) ** 2
abs_error = (d - d.median()).abs()
log_d = np.log(d)
sq_log_error = (log_d - log_d.mean()) ** 2

print(f"\nTrips over 5 hours are {tail.mean():.3%} of the data, and they contribute:")
print(f" {sq_error[tail].sum() / sq_error.sum():>6.1%} of the squared error around the mean (raw minutes)")
print(f" {abs_error[tail].sum() / abs_error.sum():>6.1%} of the absolute error around the median (raw minutes)")
print(f" {sq_log_error[tail].sum() / sq_log_error.sum():>6.1%} of the squared error in LOG space")
```

mean	18.8 min
median	10.8 min
p50	10.8 min
p90	27.6 min
p99	77.9 min
p99.9	1057.2 min
p99.99	4550.0 min
max	279454.3 min (4657.6 hours)

Trips over 5 hours: 28,374 (0.288% of trips)
Trips over 24 hours: 5,202 (0.0529% of trips)
Trips over 5 hours are 0.288% of the data, and they contribute:
99.6% of the squared error around the mean (raw minutes)
34.2% of the absolute error around the median (raw minutes)
9.0% of the squared error in LOG space

With squared error on raw minutes, the ~0.3% of trips longer than five hours make up almost **all** of the error. A model trained this way would mostly be fitting forgotten rentals.

The last line shows why we model **log(duration)**: on a log scale, a trip of several thousand hours is only a few units, so these trips matter much less. They still carry far more weight than their 0.3% share, though. So we use both: the log transform shrinks the extreme values, and the Huber loss stops the remaining ones from pulling the model too much. Step 10 checks how much the Huber loss still adds.

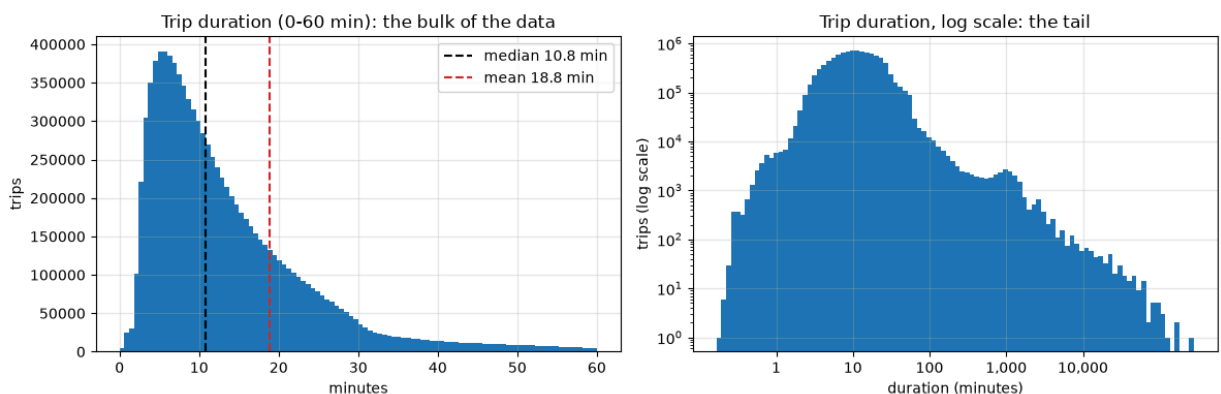
In [6]:

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].hist(d, bins=100, range=(0, 60), color="tab:blue")
axes[0].axvline(d.median(), color="black", ls="--", label=f"median {d.median():.1f} min")
axes[0].axvline(d.mean(), color="tab:red", ls="--", label=f"mean {d.mean():.1f} min")
axes[0].set_title("Trip duration (0-60 min): the bulk of the data")
axes[0].set_xlabel("minutes")
axes[0].set_ylabel("trips")
axes[0].legend()

axes[1].hist(np.log10(d), bins=100, color="tab:blue")
axes[1].set_title("Trip duration, log scale: the tail")
axes[1].set_xlabel("duration (minutes)")
axes[1].set_xticks([0, 1, 2, 3, 4], ["1", "10", "100", "1,000", "10,000"])
axes[1].set_yscale("log")
axes[1].set_ylabel("trips (log scale)")

plt.tight_layout()
plt.show()
```



6. What affects trip duration?

Our starting assumption was that three things decide how long a trip takes:

- **length**: longer routes take longer;
- **time**: the hour and day might mean more traffic, or a different kind of trip;
- **weather**: bad weather might slow riders down.

Anything else, like a rider stopping for coffee or forgetting to return the bike, can't be predicted, so we treat it as noise.

Since $\text{duration} = \text{distance} \div \text{speed}$, we can check *how* each factor works: does it change how fast people ride, or how far they go? This section uses the distance actually covered, only for exploring; the model never sees it. Trips with speeds outside 3–35 km/h (mostly forgotten bikes) are left out here.

In [7]:

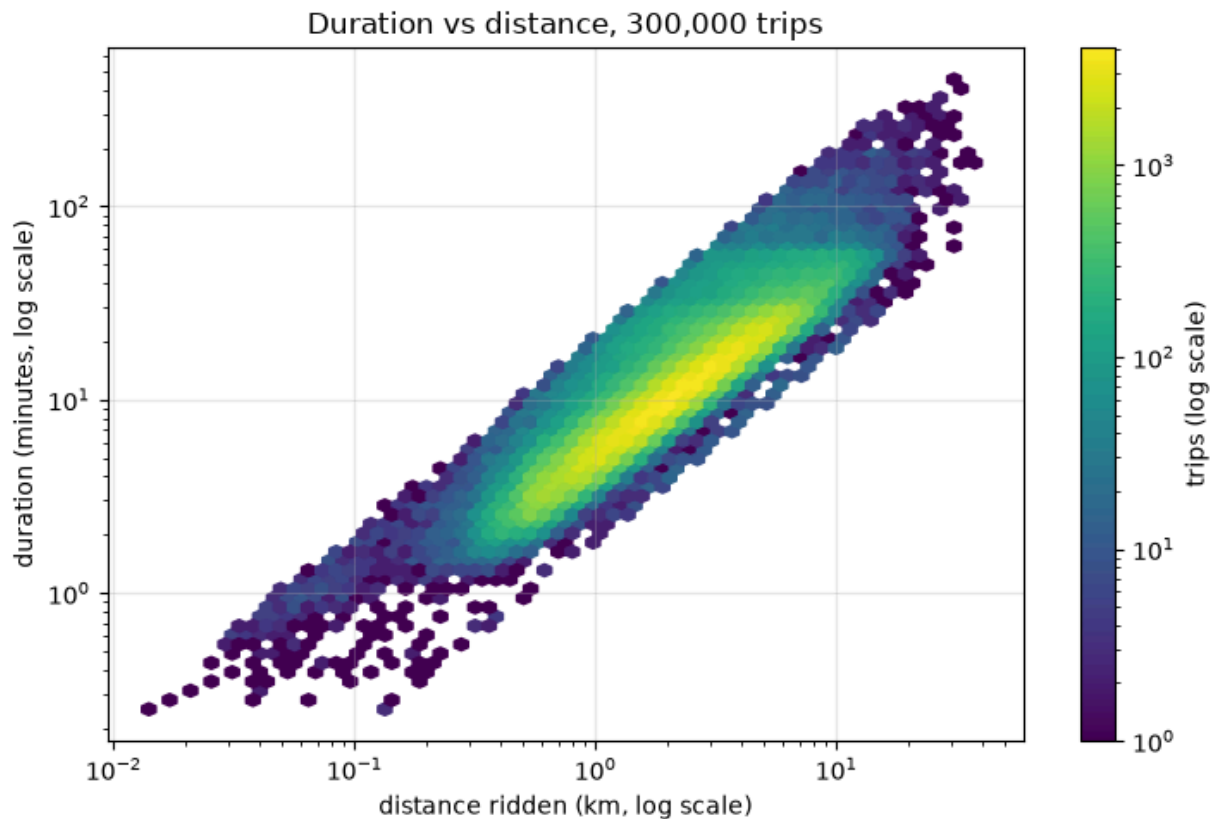
```
ride = trips[["departure", "duration_min", "distance_km", "temp_c",
"precip_mm"]].copy()
ride["speed_kmh"] = ride["distance_km"] / (ride["duration_min"] / 60)
ride = ride[ride["speed_kmh"].between(3, 35)]
print(f"Plausible rides used in this section: {len(ride):,} ({len(ride) /
len(trips):.1%} of trips)")

logs = np.log(ride[["distance_km", "speed_kmh"]])
print("\nlog(duration) = log(distance) - log(speed)")
print(f"  variance of log(distance): {logs['distance_km'].var():.3f}")
print(f"  variance of log(speed):    {logs['speed_kmh'].var():.3f}")
del logs

sample = ride.sample(300_000, random_state=RANDOM_STATE)
fig, ax = plt.subplots(figsize=(7.5, 5))
hb = ax.hexbin(sample["distance_km"], sample["duration_min"], gridsize=60,
               xscale="log", yscale="log", bins="log", mincnt=1, cmap="viridis")
ax.set_xlabel("distance ridden (km, log scale)")
ax.set_ylabel("duration (minutes, log scale)")
ax.set_title("Duration vs distance, 300,000 trips")
fig.colorbar(hb, label="trips (log scale)")
plt.tight_layout()
plt.show()
```

Plausible rides used in this section: 9,597,232 (97.6% of trips)

```
log(duration) = log(distance) - log(speed)
  variance of log(distance): 0.536
  variance of log(speed):    0.093
```

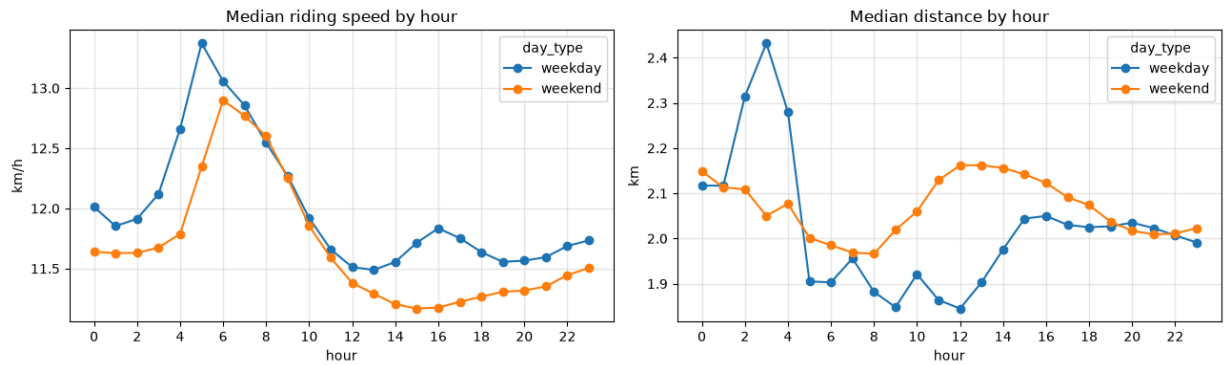


In [8]:

```
ride["hour"] = ride["departure"].dt.hour
ride["day_type"] = np.where(ride["departure"].dt.dayofweek >= 5, "weekend",
                             "weekday")

fig, axes = plt.subplots(1, 2, figsize=(13, 4))
for ax, column, title, unit in [
    (axes[0], "speed_kmh", "Median riding speed by hour", "km/h"),
    (axes[1], "distance_kmh", "Median distance by hour", "km"),
]:
    ride.pivot_table(index="hour", columns="day_type", values=column,
                      aggfunc="median").plot(ax=ax, marker="o")
    ax.set_title(title)
    ax.set_ylabel(unit)
    ax.set_xticks(range(0, 24, 2))
plt.tight_layout()
plt.show()

weekday = ride[ride["day_type"] == "weekday"]
periods = {"morning rush 7-9": [7, 8, 9], "midday 11-14": [11, 12, 13, 14],
           "evening rush 16-18": [16, 17, 18], "night 0-5": [0, 1, 2, 3, 4, 5]}
pd.DataFrame({
    label: {
        "speed (km/h)": weekday.loc[weekday["hour"].isin(hours),
        "speed_kmh"].median(),
        "distance (km)": weekday.loc[weekday["hour"].isin(hours),
        "distance_kmh"].median(),
        "duration (min)": weekday.loc[weekday["hour"].isin(hours),
        "duration_min"].median(),
    }
    for label, hours in periods.items()
}).T.rename_axis("weekdays")
```



Out[8]:

speed (km/h) distance (km) duration (min)

weekdays

morning rush 7-9	12.559	1.890	9.383
midday 11-14	11.551	1.899	10.267
evening rush 16-18	11.748	2.035	10.867
night 0-5	12.289	2.150	10.967

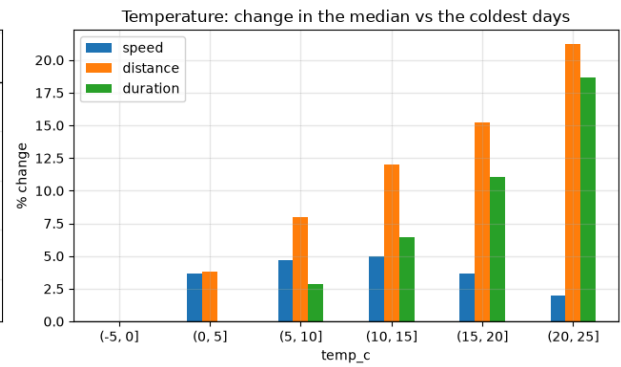
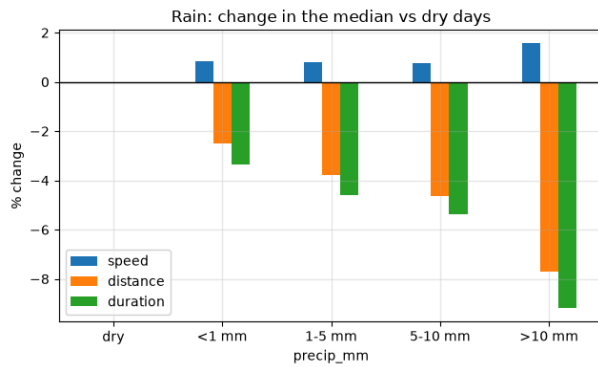
In [9]:

```
def change_vs_first_group(groups):
    medians = ride.groupby(groups, observed=True).agg(
        speed=("speed_kmh", "median"), distance=("distance_kmh", "median"),
        duration=("duration_min", "median"),
    )
    return 100 * (medians / medians.iloc[0] - 1)

rain = change_vs_first_group(pd.cut(ride["precip_mm"], [-0.01, 0.09, 1, 5, 10,
1e4],
                                labels=["dry", "<1 mm", "1-5 mm", "5-10 mm",
">10 mm"])))
temperature = change_vs_first_group(pd.cut(ride["temp_c"], np.arange(-5, 30, 5)))

fig, axes = plt.subplots(1, 2, figsize=(13, 4))
rain.plot(kind="bar", ax=axes[0])
axes[0].set_title("Rain: change in the median vs dry days")
temperature.plot(kind="bar", ax=axes[1])
axes[1].set_title("Temperature: change in the median vs the coldest days")
for ax in axes:
    ax.axhline(0, color="black", lw=1)
    ax.set_ylabel("% change")
    ax.tick_params(axis="x", rotation=0)
plt.tight_layout()
plt.show()

print("Rain, % change vs dry days:")
print(rain.round(1).to_string())
del ride, weekday, sample # only needed for this section; frees memory
```



Rain, % change vs dry days:

	speed	distance	duration
precip_mm			
dry	0.000	0.000	0.000
<1 mm	0.800	-2.500	-3.400
1-5 mm	0.800	-3.800	-4.600
5-10 mm	0.800	-4.600	-5.400
>10 mm	1.600	-7.700	-9.200

What we see:

- Length matters most.** On a log scale, distance varies several times more than speed. Most of the difference between a short and a long trip is just how far it goes.
- Time of day changes the kind of trip, not the traffic.** If traffic mattered, the rush hour would be the slowest time of day. Instead, the weekday morning is the *fastest*: commuters ride straight to work, while people riding at midday or in the afternoon are slower. So the hour tells us more about why people ride than about traffic.
- Rain doesn't slow riders down.** Median speed stays about the same (slightly higher, if anything) on rainy days, but trips get shorter. Rain changes who rides and how far, not how fast. Temperature is similar: on warm days people make longer trips, but ride at about the same speed.

Point 3 corrects our Stage 1 report, where we assumed riders are a bit slower in the rain.

This gives us something to test later. Once route length is in the model, time and weather should only add a small adjustment to speed, because most of their effect was on which trips people make. Step 11 checks this.

7. Features and data split

Route length. For each trip, we use the median distance covered by *other* trips between the same two stations. This describes the route (something a journey planner could tell you), not the trip we are predicting.

Two details make sure this doesn't leak information:

- Training trips use the other season.** A 2022 training trip gets its route length from 2023 trips only, and the other way round, so a trip's own distance is never part of its feature. Validation trips (2024) use all of 2022–2023, and test trips (2025) use all of 2022–2024.

- **Fallbacks.** A route needs at least 3 earlier trips to get its own length. Otherwise we use the median distance from the same starting station, and for a station we have never seen, the overall median.

Calendar features (hour, weekday, month) are treated as categories, because hour 23 isn't "more" than hour 9 in any useful way. One-hot encoding lets the linear model learn a separate effect for each hour. Route length goes in as a logarithm, so together with the log label the model learns $\text{duration} \propto \text{length}^b$.

The split is by season, so the model is always evaluated on a season it hasn't seen. A random split would put trips from the same days in both training and validation, and the results would look better than they really are.

Set	Seasons
Training	2022, 2023
Validation	2024
Test	2025

The Huber solver is slow on millions of rows, so all models are trained on the **same random sample** of 400,000 training trips. Validation and test always use every trip in their season. The test features are only built in step 13.

In [10]:

```
CATEGORICAL = ["hour", "weekday", "month"]
NUMERIC = ["log_route", "round_trip", "temp_c", "precip_mm"]
FEATURES = CATEGORICAL + NUMERIC
ROUTE_MIN_TRIPS = 3

season = trips["departure"].dt.year

def route_length(df, history):
    # Typical route length for each trip in df, using only the trips in `history`.
    # Routes with too few trips fall back to the starting station's median, then
    # the overall median.
    by_route = history.groupby("route")["distance_m"].agg(["median", "size"])
    by_route = by_route.loc[by_route["size"] >= ROUTE_MIN_TRIPS, "median"]
    by_station = history.groupby("from_code")["distance_m"].median()

    length = df["route"].map(by_route)
    source = pd.Series("route", index=df.index)
    source[length.isna()] = "station"
    length = length.fillna(df["from_code"].map(by_station))
    source[length.isna()] = "overall"
    length = length.fillna(history["distance_m"].median())
    return length.astype("float64"), source

def cross_fitted_route_length(df):
    # For training data: each season gets its route lengths from the other seasons
    # only,
    # so a trip's own distance is never used to build its own feature.
```

```

df_season = df["departure"].dt.year
length = pd.Series(np.nan, index=df.index)
source = pd.Series("", index=df.index, dtype=object)
for s in sorted(df_season.unique()):
    rows = df_season == s
    s_length, s_source = route_length(df[rows], history=df[~rows])
    length.loc[s_length.index] = s_length
    source.loc[s_source.index] = s_source
return length, source

def make_features(df, length):
    return pd.DataFrame({
        "hour": df["departure"].dt.hour.astype("int16"),
        "weekday": df["departure"].dt.dayofweek.astype("int16"),
        "month": df["departure"].dt.month.astype("int16"),
        "log_route": np.log(length),
        "round_trip": df["round_trip"].astype("int8"),
        "temp_c": df["temp_c"].astype("float64"),
        "precip_mm": df["precip_mm"].astype("float64"),
    }, index=df.index)[FEATURES]

train_rows = trips[season <= 2023]
val_rows = trips[season == 2024]

route_train, source_train = cross_fitted_route_length(train_rows)
route_val, source_val = route_length(val_rows, history=train_rows)

X_train_full, y_train_full = make_features(train_rows, route_train),
train_rows["duration_min"]
X_val, y_val = make_features(val_rows, route_val), val_rows["duration_min"]

sample = X_train_full.sample(TRAIN_SAMPLE, random_state=RANDOM_STATE).index
X_train, y_train = X_train_full.loc[sample], y_train_full.loc[sample]
print(f"training (full) {len(X_train_full):>10,} trips")
print(f"training sample {len(X_train):>10,} trips")
print(f"validation      {len(X_val):>10,} trips")
del X_train_full, y_train_full, route_train

coverage = pd.DataFrame({
    "training (cross-fitted)": source_train.value_counts(normalize=True),
    "validation": source_val.value_counts(normalize=True),
}).mul(100).reindex(["route", "station", "overall"]).fillna(0)
print("\nWhere each trip's route length came from (% of trips):")
print(coverage.round(2).to_string())
X_train.head()

```

```

training (full)  4,876,755 trips
training sample  400,000 trips
validation       2,508,741 trips
Where each trip's route length came from (% of trips):
      training (cross-fitted)  validation
route                97.440         96.480
station              2.560         3.520
overall              0.000         0.000

```

Out[10]:

	hour	weekday	month	log_route	round_trip	temp_c	precip_mm
68167	17	5	4	8.685	0	7.900	0.000
186345	8	1	5	7.343	0	11.600	3.600
2307348	9	4	10	6.714	0	10.300	3.400
3835782	17	0	7	7.981	0	15.900	0.000
923465	18	0	6	8.058	0	14.100	0.000

8. Train and compare the models

We train three models. Each pair differs in only one thing, so each comparison answers one question:

Model	Model type	Loss
Linear, squared error	linear	squared error
Linear, Huber	linear	Huber
Boosted trees, absolute error	non-linear	absolute error

- **Squared error vs Huber** (same model type): does the robust loss still help once we use $\log(\text{duration})$?
- **Linear (Huber) vs boosted trees** (both robust losses): are there interactions the linear model misses, like rain mattering more on long routes?

The Huber loss works like squared error for small errors and like absolute error for large ones (threshold $\epsilon = 1.35$). The trees use absolute error because scikit-learn's histogram-based boosting has no Huber option; in our earlier tests the two robust losses gave almost the same results. All three models are trained on $\log(\text{duration})$, and their predictions are turned back into minutes.

We also include two simple baselines, to see how much the models actually learn:

- **Constant median:** the same prediction for every trip.
- **Physics:** route length $\times$ one typical pace (minutes per km), taken from the training data. This is our "distance $\div$ speed" idea without any machine learning. If the models only just beat it, learning adds little on top of simple arithmetic.

Metrics (all in minutes):

- **MAE:** mean absolute error.
- **MedAE:** median absolute error, i.e. the error on a typical trip. Forgotten rentals don't affect it.
- **Within 2 min:** the share of trips predicted within two minutes, which is what matters most to a rider.

We fixed the selection rule before looking at any results: **the model with the lowest validation MAE is chosen** for the final test. We don't use RMSE or R^2 , because with this label they mostly measure the forgotten rentals.

In [11]:

```
class ConstantMedian:
    # Baseline: predicts the training median for every trip.
    def fit(self, X, y):
        self.value_ = float(np.median(y))
        return self

    def predict(self, X):
        return np.full(len(X), self.value_)

class PhysicsBaseline:
    # Baseline: route length times one typical pace (minutes per km). No learning
    # involved.
    def fit(self, X, y):
        route_km = np.exp(X["log_route"]) / 1000
        self.min_per_km_ = float(np.median(y / route_km))
        return self

    def predict(self, X):
        return np.exp(X["log_route"]).to_numpy() / 1000 * self.min_per_km_

def linear_model(regressor, features=FEATURES):
    cats = [f for f in features if f in CATEGORICAL]
    nums = [f for f in features if f not in CATEGORICAL]
    parts = []
    if cats:
        parts.append(("cat", OneHotEncoder(handle_unknown="ignore"), cats))
    if nums:
        parts.append(("num", StandardScaler(), nums))
    return make_pipeline(ColumnTransformer(parts), regressor)

def on_log_scale(model):
    # Train on log(duration), but return predictions in minutes.
    return TransformedTargetRegressor(regressor=model, func=np.log,
    inverse_func=np.exp)

def huber():
    return HuberRegressor(epsilon=1.35, alpha=1e-4, max_iter=500)

def build_models():
    return {
        "Linear, squared error": on_log_scale(linear_model(LinearRegression())),
        "Linear, Huber": on_log_scale(linear_model(huber())),
        "Boosted trees, absolute error":
on_log_scale(HistGradientBoostingRegressor(
    loss="absolute_error", categorical_features=CATEGORICAL,
    random_state=RANDOM_STATE,
    )),
    }
```

```

def build_references():
    return {
        "Constant: median": ConstantMedian(),
        "Physics: route length x typical pace": PhysicsBaseline(),
    }

def evaluate(y_true, y_pred):
    err = np.abs(np.asarray(y_true, dtype="float64") - np.asarray(y_pred,
dtype="float64"))
    return {"MAE": err.mean(), "MedAE": np.median(err), "within 2 min %": 100 *
(err <= 2).mean()}

def report_solver_warnings(caught):
    messages = sorted({str(w.message).splitlines()[0] for w in caught})
    print("Solver warnings:", "none" if not messages else "")
    for m in messages:
        print("  -", m)

```

In [12]:

```

references, models = build_references(), build_models()
val_pred, rows = {}, []

with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    for name, model in {**references, **models}.items():
        model.fit(X_train, y_train)
        val_pred[name] = model.predict(X_val)
        rows.append({"model": name, **evaluate(y_val, val_pred[name])})
report_solver_warnings(caught)

results = pd.DataFrame(rows).set_index("model")
baseline_medae = results.loc["Constant: median", "MedAE"]
results["MedAE gain vs constant %"] = 100 * (baseline_medae - results["MedAE"]) /
baseline_medae

physics = references["Physics: route length x typical pace"]
print(f"Physics reference: {physics.min_per_km:.2f} min per km of route "
      f"(= {60 / physics.min_per_km:.2f} km/h)")

best_name = results.loc[list(models), "MAE"].idxmin()
print(f"Selected by the pre-declared rule (lowest validation MAE of the three
models): {best_name}")
results

```

Solver warnings: none

Physics reference: 5.23 min per km of route (= 11.48 km/h)

Selected by the pre-declared rule (lowest validation MAE of the three models):

Linear, Huber

Out[12]:

	MAE	MedAE	within 2 min %	MedAE gain vs constant %
model				
Constant: median	13.978	5.300	18.984	0.000
Physics: route length x typical pace	11.219	2.059	49.131	61.145
Linear, squared error	11.002	2.205	46.099	58.403
Linear, Huber	10.909	1.857	52.461	64.956
Boosted trees, absolute error	10.955	1.853	52.456	65.041

In [13]:

```
# Training error = error on the same 400,000 trips each model was trained on.
# A much lower training error than validation error would suggest overfitting.
training = pd.DataFrame(
    [{"model": name, **evaluate(y_train, model.predict(X_train))}] for name, model
    in (**references, **models).items()
).set_index("model")
metrics = ["MAE", "MedAE", "within 2 min %"]
pd.concat({"training": training[metrics], "validation": results[metrics]}, axis=1)
```

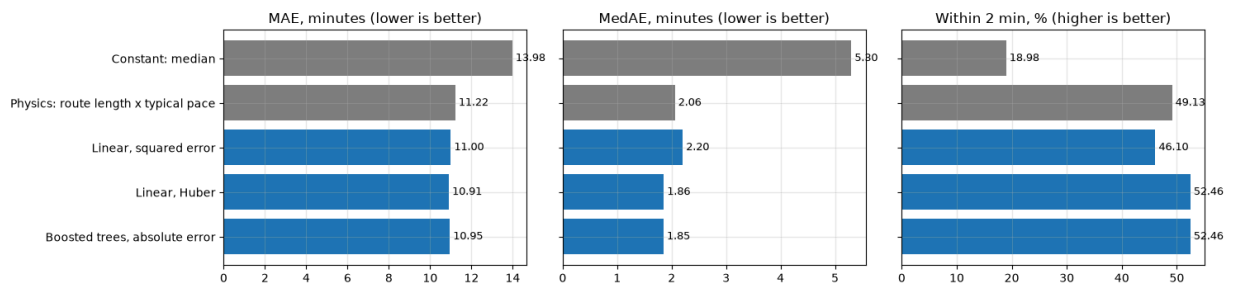
Out[13]:

	training			validation		
	MAE	MedAE	within 2 min %	MAE	MedAE	within 2 min %
model						
Constant: median	10.404	4.883	20.251	13.978	5.300	18.984
Physics: route length x typical pace	7.921	1.755	54.063	11.219	2.059	49.131
Linear, squared error	7.736	1.983	50.362	11.002	2.205	46.099
Linear, Huber	7.622	1.617	57.696	10.909	1.857	52.461
Boosted trees, absolute error	7.570	1.576	58.328	10.955	1.853	52.456

In [14]:

```
fig, axes = plt.subplots(1, 3, figsize=(15, 3.6), sharey=True)
colors = ["tab:gray" if name in references else "tab:blue" for name in
results.index]
for ax, column, title in zip(
    axes,
    ["MAE", "MedAE", "within 2 min %"],
    ["MAE, minutes (lower is better)", "MedAE, minutes (lower is better)", "Within
2 min, % (higher is better)"],
):
    ax.barh(results.index, results[column], color=colors)
    for y, v in enumerate(results[column]):
        ax.text(v, y, f" {v:.2f}", va="center", fontsize=9)
    ax.set_title(title)
```

```
axes[0].invert_yaxis()
plt.tight_layout()
plt.show()
```



What the table shows:

- Route length does most of the work.** The physics baseline (route length $\times$ 5.23 min/km, i.e. 11.48 km/h, no learning) already cuts the typical error from 5.30 to 2.06 minutes. That's 61 of the 65 percentage points of improvement the best model gets.
- The loss still matters, even with log(duration).** On typical trips, the squared-error model is *worse than the physics baseline*: MedAE 2.205 vs 2.059 minutes, and 46.1% vs 49.1% of trips within 2 minutes (Huber gets 52.5%). The remaining long trips pull its predictions up; in step 10, its average prediction is about half a minute higher than Huber's.
- Linear and trees perform the same.** Huber and the boosted trees differ by 0.05 minutes in MAE and 0.004 minutes in MedAE, and predict the same share of trips within 2 minutes. Even with route length to work with, the trees don't find anything the linear model misses.

Training vs validation. Every model does better on its training trips than on 2024, including the constant and the physics baseline, which fit at most one number (the constant's MedAE is 4.88 on training vs 5.30 on validation). So the gap mostly comes from 2024 being a harder season, not from overfitting. The trees have the lowest training error (MedAE 1.58) but no advantage on validation, a small sign that their extra flexibility fits the training data rather than the real pattern.

Using the rule we fixed beforehand (lowest validation MAE), we choose **Linear, Huber** for the final test.

9. Are the differences real?

Some differences between the models are only a fraction of a minute, so we check whether they would hold up on different data. We resample the 2024 validation season 2,000 times, picking **whole days** instead of single trips, since trips on the same day share the weather and aren't independent.

Each time, we compare two models on the same resampled days. This gives a 95% interval for the difference between them. If the interval doesn't include zero, the difference is real.

```

COMPARISONS = [
    # (question, first model, second model, short name of first, short name of
    second)
    ("loss", "Linear, Huber", "Linear, squared error", "Huber", "squared"),
    ("hypothesis space", "Boosted trees, absolute error", "Linear, Huber",
    "trees", "linear"),
    ("learning vs physics", best_name, "Physics: route length x typical pace",
    "model", "physics"),
]

def verdict(low, high, first, second, lower_is_better):
    if low <= 0 <= high:
        return "no clear difference"
    first_wins = high < 0 if lower_is_better else low > 0
    return f"{first if first_wins else second} better"

def paired_bootstrap(y_true, predictions, dates, n_resamples=2000,
seed=RANDOM_STATE):
    # 95% intervals for the difference between two models, resampling whole days.
    day_index, days = pd.factorize(dates)
    n_days = len(days)
    y_true = np.asarray(y_true, dtype="float64")
    trips_per_day = np.bincount(day_index, minlength=n_days)

    def per_day(values):
        return np.bincount(day_index, weights=values, minlength=n_days)

    # Sum the errors per day once; each resample then only has to add up day
    totals.
    abs_err = {name: per_day(np.abs(y_true - p)) for name, p in
    predictions.items()}
    within = {name: per_day((np.abs(y_true - p) <= 2).astype("float64")) for name,
    p in predictions.items()}

    # Each row is one resample: a list of days drawn with replacement.
    draws = np.random.default_rng(seed).integers(0, n_days, size=(n_resamples,
    n_days))
    boot_trips = trips_per_day[draws].sum(axis=1)

    def boot(by_day):
        return by_day[draws].sum(axis=1) / boot_trips

    def overall(by_day):
        return by_day.sum() / trips_per_day.sum()

    rows = []
    for question, a, b, short_a, short_b in COMPARISONS:
        mae_lo, mae_hi = np.percentile(boot(abs_err[a]) - boot(abs_err[b]), [2.5,
    97.5])
        w_lo, w_hi = np.percentile(100 * (boot(within[a]) - boot(within[b])),
    [2.5, 97.5])
        rows.append({
            "question": question,
            "comparison": f"{short_a} - {short_b}",
            "MAE diff (min)": overall(abs_err[a]) - overall(abs_err[b]),
            "MAE 95% CI": f"[{mae_lo:+.3f}, {mae_hi:+.3f}]",
            "MAE verdict": verdict(mae_lo, mae_hi, short_a, short_b,

```

```

lower_is_better=True),
    "within-2 diff (pp)": 100 * (overall(within[a]) - overall(within[b])),
    "within-2 95% CI": f"[{w_lo:+.2f}, {w_hi:+.2f}]",
    "within-2 verdict": verdict(w_lo, w_hi, short_a, short_b,
lower_is_better=False),
    })
print(f"{n_days} days, {n_resamples:,} day-level resamples")
return pd.DataFrame(rows).set_index("question")

paired_bootstrap(y_val, val_pred, val_rows["departure"].dt.normalize())

```

214 days, 2,000 day-level resamples

Out[15]:

	comparison	MAE diff (min)	MAE 95% CI	MAE verdict	within-2 diff (pp)	within-2 95% CI	within-2 verdict
question							
loss	Huber - squared	-0.093	[-0.098, -0.088]	Huber better	6.362	[+6.14, +6.57]	Huber better
	trees - linear	0.046	[+0.043, +0.049]	linear better	-0.005	[-0.07, +0.06]	no clear difference
learning vs physics	model - physics	-0.310	[-0.317, -0.302]	model better	3.330	[+3.12, +3.54]	model better

- **Loss: Huber is clearly better.** Both intervals are far from zero: 0.093 minutes lower MAE, and **6.4 percentage points more trips within 2 minutes**. The second number matters more to a rider: with Huber, about one extra trip in sixteen is predicted within two minutes.
- **Linear vs trees: no real difference.** The linear model's MAE is "significantly" lower, but only by 0.046 minutes, under three seconds. With 2.5 million trips, even tiny differences give tight intervals. For trips within 2 minutes, there's no difference at all. A difference can be statistically clear and still not matter in practice.
- **Model vs physics: a real but small gain.** The model beats the simple rule by 0.31 minutes of MAE and 3.3 percentage points within 2 minutes.

These intervals only cover variation within 2024. They don't show how much results change from one year to the next; the 2025 test in step 13 checks that.

10. Does the loss still matter?

Using $\log(\text{duration})$ already reduces the effect of the forgotten rentals, so does the Huber loss still add anything? To check, we retrain each model without the longest 0.1% of training trips and see how much its predictions change on 200,000 validation trips.

Retraining always changes a model a little, even if nothing special is removed. Boosted trees, for example, use a random internal split for early stopping. So as a control, we also retrain

after removing the **same number of random trips**. The effect of the outliers is the difference between the two.

In [16]:

```
cutoff = y_train.quantile(0.999)
longest = y_train > cutoff
n_removed = int(longest.sum())
random_drop = y_train.sample(n_removed, random_state=RANDOM_STATE + 1).index #
control group
keep_trimmed = ~longest
keep_random = ~y_train.index.isin(random_drop)
probe = X_val.sample(200_000, random_state=RANDOM_STATE)
print(f"Removed {n_removed:,} training trips longer than {cutoff:,.0f} min
({n_removed / len(y_train):.2%}); "
      f"control: {n_removed:,} trips removed at random")

def mean_shift_pct(new, reference):
    return 100 * (np.abs(new - reference) / reference).mean()

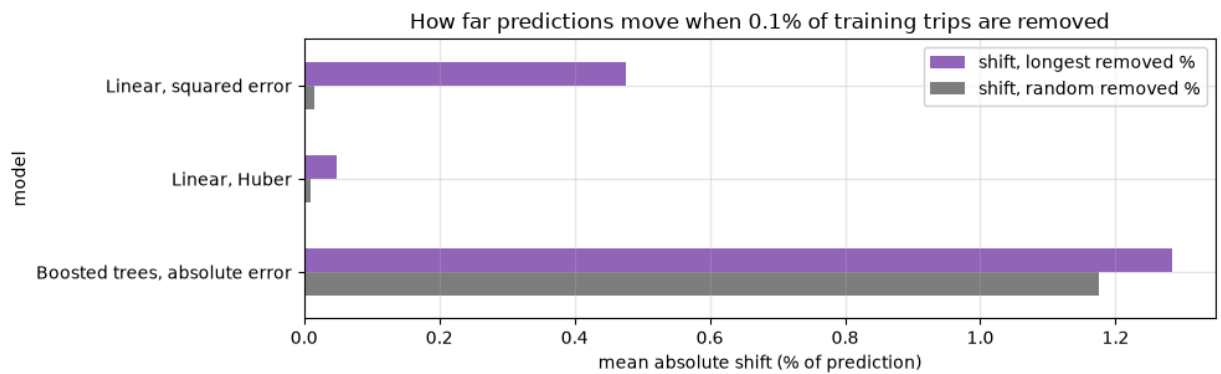
rows = []
with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    for name in models:
        full = models[name].predict(probe)
        trimmed = build_models()[name].fit(X_train[keep_trimmed],
y_train[keep_trimmed]).predict(probe)
        control = build_models()[name].fit(X_train[keep_random],
y_train[keep_random]).predict(probe)
        rows.append({
            "model": name,
            "mean prediction (min)": full.mean(),
            "shift, longest removed %": mean_shift_pct(trimmed, full),
            "shift, random removed %": mean_shift_pct(control, full),
        })
report_solver_warnings(caught)

sensitivity = pd.DataFrame(rows).set_index("model")
sensitivity["outlier effect (pp)"] = sensitivity["shift, longest removed %"] -
sensitivity["shift, random removed %"]

ax = sensitivity[["shift, longest removed %", "shift, random removed %"]].plot(
    kind="barh", figsize=(10, 3.2), color=["tab:purple", "tab:gray"])
ax.set_title("How far predictions move when 0.1% of training trips are removed")
ax.set_xlabel("mean absolute shift (% of prediction)")
ax.invert_yaxis()
plt.tight_layout()
plt.show()
sensitivity
```

Removed 400 training trips longer than 1,016 min (0.10%); control: 400 trips removed at random

Solver warnings: none



Out[16]:

	mean prediction (min)	shift, longest removed %	shift, random removed %	outlier effect (pp)
model				
Linear, squared error	12.034	0.476	0.015	0.461
Linear, Huber	11.486	0.048	0.009	0.039
Boosted trees, absolute error	11.289	1.284	1.175	0.109

The control changes how to read this:

- **Boosted trees** move 1.28% when the longest trips are removed, but also 1.18% when random trips are removed. So most of that change is just normal retraining noise. The real outlier effect is only 0.11 percentage points.
- **Squared error** moves 0.48%, against 0.015% in the control. Its outlier effect (0.46 points) is **about 12 times bigger than Huber's** (0.04 points).

In an earlier version of this experiment on raw minutes, squared-error predictions moved by 17–23%. So the log transform removes most of the instability, and Huber removes most of what's left.

With log(duration), the bigger problem with squared error isn't instability but **bias**: its average prediction is 12.03 minutes vs 11.49 for Huber, about half a minute too high on normal trips. That's why it loses 6 percentage points of trips within 2 minutes (step 9). So the log transform and the Huber loss each fix a different part of the problem.

11. Which features matter?

This tests the assumption from step 6 directly. We retrain the Huber model with different combinations of features and see what each group adds:

- **Length**: route length and the round-trip flag (both describe the route);
- **Time**: hour, weekday, month;
- **Weather**: temperature and precipitation.

The first row uses the Stage 1 features (time and weather, no route) for comparison.

In [17]:

```
ABLATION = {
    "Time + weather (Stage 1 features)": ["hour", "weekday", "month", "temp_c",
    "precip_mm"],
    "Length": ["log_route", "round_trip"],
    "Length + time": ["log_route", "round_trip", "hour", "weekday", "month"],
    "Length + weather": ["log_route", "round_trip", "temp_c", "precip_mm"],
    "Length + time + weather": FEATURES,
}

rows = [{"features": "None (constant median)", **evaluate(y_val,
val_pred["Constant: median"])}]
with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    for label, features in ABLATION.items():
        if features == FEATURES:
            pred = val_pred["Linear, Huber"] # already trained in step 8
        else:
            pred = on_log_scale(linear_model(huber(), features)).fit(X_train,
y_train).predict(X_val)
        rows.append({"features": label, **evaluate(y_val, pred)})
report_solver_warnings(caught)

ablation = pd.DataFrame(rows).set_index("features")
none_medae = ablation.loc["None (constant median)", "MedAE"]
full_medae = ablation.loc["Length + time + weather", "MedAE"]
ablation["share of total MedAE gain %"] = 100 * (none_medae - ablation["MedAE"]) /
(none_medae - full_medae)
ablation
```

Solver warnings: none

Out[17]:

	MAE	MedAE	within 2 min %	share of total MedAE gain %
features				
None (constant median)	13.978	5.300	18.984	0.000
Time + weather (Stage 1 features)	13.932	5.250	19.104	1.458
Length	10.950	1.865	52.326	99.778
Length + time	10.913	1.850	52.587	100.225
Length + weather	10.945	1.875	52.165	99.501
Length + time + weather	10.909	1.857	52.461	100.000

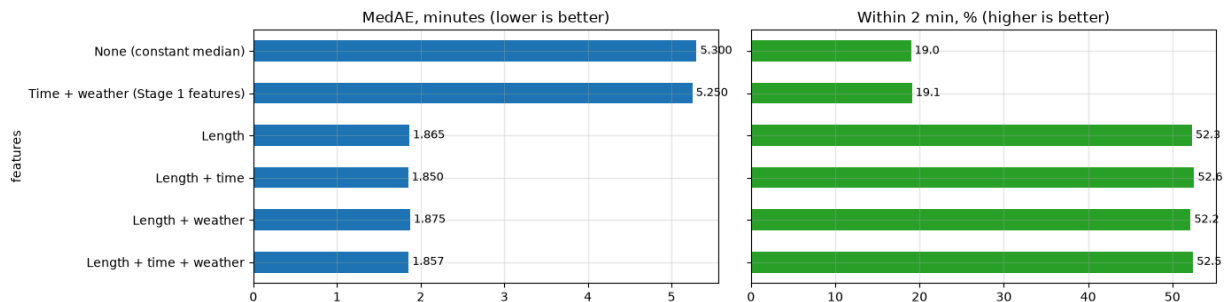
In [18]:

```
fig, axes = plt.subplots(1, 2, figsize=(14, 3.6), sharey=True)
ablation["MedAE"].plot(kind="barh", ax=axes[0], color="tab:blue")
axes[0].set_title("MedAE, minutes (lower is better)")
ablation["within 2 min %"].plot(kind="barh", ax=axes[1], color="tab:green")
axes[1].set_title("Within 2 min, % (higher is better)")
axes[0].invert_yaxis()
for ax, column, fmt in [(axes[0], "MedAE", "{:.3f}"), (axes[1], "within 2 min %",
"{:.1f}")]:
```

```

for y, v in enumerate(ablation[column]):
    ax.text(v, y, " " + fmt.format(v), va="center", fontsize=9)
plt.tight_layout()
plt.show()

```



Checking the three assumptions from step 6:

- **Length: confirmed, and by far the most important.** Route length alone gives 99.8% of the total improvement: MedAE drops from 5.30 to 1.865 minutes, and trips within 2 minutes go from 19.0% to 52.3%.
- **Time: confirmed, but small.** On top of length, the calendar features improve all three metrics a little: MedAE 1.865 → 1.850, +0.26 percentage points within 2 minutes, −0.037 minutes MAE.
- **Weather: not supported.** Adding weather to length makes the typical-trip metrics slightly *worse* (MedAE 1.865 → 1.875, −0.16 points within 2 minutes) and improves MAE by only 0.005 minutes. With daily weather, there's no sign that it helps.

The first row, the Stage 1 features (time and weather without the route), gets only **1.5%** of the improvement. That's the main reason we added the route in Stage 2.

By MedAE, length + time is slightly better than the full model. We still keep the full model because the features and the selection rule were fixed before seeing these results. Dropping weather now would mean picking features based on the same data we report.

12. What the model learned

The model is linear in $\log(\text{duration})$, so each coefficient can be turned into a percentage change in duration. These come from the Huber model trained in step 8.

In [19]:

```

fitted = models["Linear, Huber"].regressor_
pre = fitted.named_steps["columntransformer"]
coef = pd.Series(fitted.named_steps["huberregressor"].coef_,
index=pre.get_feature_names_out())
numeric_names = [f for f in FEATURES if f not in CATEGORICAL]
# The numeric features were standardised, so divide by their scale to get the
effect
# per original unit (per °C, per mm, ...).
per_unit = coef[[f"num_{n}" for n in numeric_names]].to_numpy() /
pre.named_transformers_["num"].scale_
per_unit = pd.Series(per_unit, index=numeric_names)

```

```

def pct(log_effect):
    # An effect on log(duration) -> % change in duration.
    return 100 * (np.exp(log_effect) - 1)

b = per_unit["log_route"]
effects = pd.Series({
    "route length exponent b (duration ~ length^b)": b,
    "doubling the route length: % longer trip": pct(b * np.log(2)),
    "round trip vs one-way: %": pct(per_unit["round_trip"]),
    "+1 C temperature: %": pct(per_unit["temp_c"]),
    "+1 mm/day rain: %": pct(per_unit["precip_mm"]),
    "+10 mm/day rain: %": pct(10 * per_unit["precip_mm"]),
})
print(effects.round(3).to_string())

hours = coef[[n for n in coef.index if n.startswith("cat__hour_")]]
hours.index = [int(n.rsplit("_", 1)[1]) for n in hours.index]
# One-hot effects only mean something relative to each other, so centre them on
their average.
hours = pct(hours.sort_index() - hours.mean())

weekdays = coef[[n for n in coef.index if n.startswith("cat__weekday_")]]
weekdays.index = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
weekdays = pct(weekdays - weekdays.mean())

print("\nHour of day, % change in duration vs the daily average (same route):")
for start in (0, 12):
    print(" " + " ".join(f"{h:02d}h {hours[h]:+5.1f}" for h in range(start,
start + 12)))
print("Weekday, % change: " + " ".join(f"{d} {v:+.1f}" for d, v in
weekdays.items()))

fig, axes = plt.subplots(1, 2, figsize=(14, 3.8), gridspec_kw={"width_ratios": [3,
1]})
axes[0].bar(hours.index, hours.values, color=np.where(hours.values < 0,
"tab:green", "tab:red"))
axes[0].set_title("Hour of day: % change in duration vs the daily average (same
route)")
axes[0].set_xlabel("hour")
axes[0].set_xticks(range(0, 24, 2))
axes[1].bar(weekdays.index, weekdays.values, color=np.where(weekdays.values < 0,
"tab:green", "tab:red"))
axes[1].set_title("Weekday: % change")
for ax in axes:
    ax.axhline(0, color="black", lw=1)
    ax.set_ylabel("%")
plt.tight_layout()
plt.show()

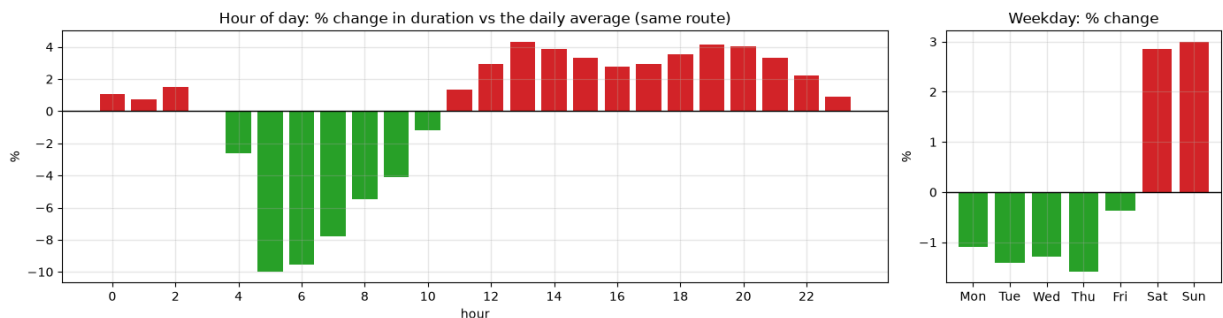
```

route length exponent b (duration $\sim \text{length}^b$)	0.847
doubling the route length: % longer trip	79.926
round trip vs one-way: %	104.982
+1 C temperature: %	0.236
+1 mm/day rain: %	-0.120
+10 mm/day rain: %	-1.190

Hour of day, % change in duration vs the daily average (same route):

00h	+1.1	01h	+0.7	02h	+1.5	03h	-0.1	04h	-2.6	05h	-10.0	06h	-9.5	07h	-7.8	08h	-5.5	09h	-4.1	10h	-1.2	11h	+1.4	12h	+2.9	13h	+4.3	14h	+3.9	15h	+3.3	16h	+2.8	17h	+2.9	18h	+3.6	19h	+4.1	20h	+4.0	21h	+3.3	22h	+2.2	23h	+0.9
-----	------	-----	------	-----	------	-----	------	-----	------	-----	-------	-----	------	-----	------	-----	------	-----	------	-----	------	-----	------	-----	------	-----	------	-----	------	-----	------	-----	------	-----	------	-----	------	-----	------	-----	------	-----	------	-----	------	-----	------

Weekday, % change: Mon -1.1 Tue -1.4 Wed -1.3 Thu -1.6 Fri -0.4 Sat +2.9 Sun +3.0



All the effects below are for the same route, so they are really about speed:

- **Route length: duration $\propto \text{length}^{0.85}$.** Doubling the route makes a trip 80% longer, not twice as long. Longer trips are ridden faster, probably because there are fewer junctions and stops per kilometre.
- **Round trips take about twice as long** (+105%) as one-way trips of the same typical length. Riding back to where you started is usually a leisure ride with stops.
- **Hour of day covers about 14 percentage points.** Trips are fastest early in the morning and during the morning commute (−10% at 05:00, −5.5% at 08:00) and slowest in the early afternoon and evening (+4.3% at 13:00, +4.1% at 19:00). This matches step 6: the hour reflects why people ride, not traffic.
- **Weekends are slower:** Saturday and Sunday about +3%, Monday to Thursday about −1% to −1.6%, so about 4–5% between them.
- **Temperature:** +0.24% per °C, so a day that is 20 °C warmer means trips about 5% longer (more relaxed riding).
- **Rain:** −0.12% per mm, so 10 mm of rain makes the same route about 1.2% *faster*. This is the opposite of what we assumed in Stage 1, and it matches step 6: people hurry in the rain.

These effects are real and easy to interpret, but they are small compared with the differences between individual riders. That's why time and weather add so little to accuracy (step 11).

13. Test on the 2025 season

The model was chosen using 2024. Now all models are retrained on 2022–2024, with route lengths recomputed from those seasons (again using the other seasons for each training trip), and tested **once** on 2025. We show all three models and mark the chosen one.

Some stations opened in 2025 and have no history, so their trips fall back to the station or overall median length.

In [20]:

```
trainval_rows = trips[season <= 2024]
test_rows = trips[season == 2025]

route_tv, _ = cross_fitted_route_length(trainval_rows)
route_test, source_test = route_length(test_rows, history=trainval_rows)

X_tv_full, y_tv_full = make_features(trainval_rows, route_tv),
trainval_rows["duration_min"]
X_test, y_test = make_features(test_rows, route_test), test_rows["duration_min"]
sample_tv = X_tv_full.sample(TRAIN_SAMPLE, random_state=RANDOM_STATE).index
X_tv, y_tv = X_tv_full.loc[sample_tv], y_tv_full.loc[sample_tv]
del X_tv_full, y_tv_full, route_tv

print(f"Test season 2025: {len(X_test):,} trips")
print("Where each test trip's route length came from (% of trips):")
print((100 * source_test.value_counts(normalize=True)).reindex(["route",
"station", "overall"]).fillna(0).round(2).to_string())

test_pred, rows = {}, []
with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    for name, model in {**build_references(), **build_models()}.items():
        model.fit(X_tv, y_tv)
        test_pred[name] = model.predict(X_test)
        rows.append({"model": name, **evaluate(y_test, test_pred[name])})
report_solver_warnings(caught)

test_results = pd.DataFrame(rows).set_index("model")
test_results["selected on 2024"] = np.where(test_results.index == best_name, "<-
selected", "")
test_results
```

Test season 2025: 2,451,856 trips
Where each test trip's route length came from (% of trips):
route 97.400
station 2.240
overall 0.350
Solver warnings: none

Out[20]:

	MAE	MedAE	within 2 min %	selected on 2024
model				
Constant: median	14.892	5.433	18.610	
Physics: route length x typical pace	11.949	2.140	48.022	
Linear, squared error	11.704	2.353	43.345	
Linear, Huber	11.548	1.911	51.526	<- selected
Boosted trees, absolute error	11.599	1.890	51.813	

In [21]:

```
# Same comparisons as in step 9, now on the test season.
paired_bootstrap(y_test, test_pred, test_rows["departure"].dt.normalize())
```

214 days, 2,000 day-level resamples

Out[21]:

	comparison	MAE diff (min)	MAE 95% CI	MAE verdict	within-2 diff (pp)	within-2 95% CI	within-2 verdict
question							
loss	Huber - squared	-0.156	[-0.163, -0.150]	Huber better	8.182	[+7.88, +8.47]	Huber better
hypothesis space	trees - linear	0.051	[+0.046, +0.057]	linear better	0.286	[+0.20, +0.37]	trees better
learning vs physics	model - physics	-0.401	[-0.409, -0.391]	model better	3.504	[+3.23, +3.75]	model better

In [22]:

```
chosen = pd.DataFrame({
    "actual": y_test.to_numpy(),
    "predicted": test_pred[best_name],
    "route_km": np.exp(X_test["log_route"].to_numpy()) / 1000,
    "hour": X_test["hour"].to_numpy(),
})
chosen["route band"] = pd.qcut(chosen["route_km"], 10, duplicates="drop")

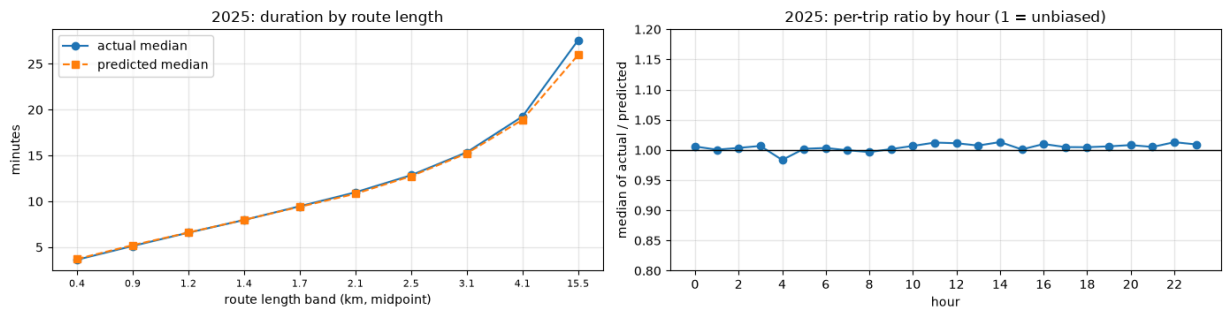
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
by_route = chosen.groupby("route band", observed=True)[["actual",
    "predicted"]].median()
axes[0].plot(range(len(by_route)), by_route["actual"], marker="o", label="actual
median")
axes[0].plot(range(len(by_route)), by_route["predicted"], marker="s", ls="--",
label="predicted median")
axes[0].set_xticks(range(len(by_route)))
axes[0].set_xticklabels([f"{iv.mid:.1f}" for iv in by_route.index], fontsize=8)
axes[0].set_xlabel("route length band (km, midpoint)")
axes[0].set_ylabel("minutes")
axes[0].set_title("2025: duration by route length")
axes[0].legend()

# Compare actual/predicted per trip instead of comparing medians. Each hour mixes
short and
# long routes, and actual times spread further above the prediction than below it,
so the
# median actual time is higher than the median prediction even if the predictions
are unbiased.
ratio = chosen["actual"] / chosen["predicted"]
ratio_by_hour = ratio.groupby(chosen["hour"]).median()
axes[1].plot(ratio_by_hour.index, ratio_by_hour.values, marker="o")
axes[1].axhline(1, color="black", lw=1)
axes[1].set_ylim(0.8, 1.2)
axes[1].set_xlabel("hour")
axes[1].set_xticks(range(0, 24, 2))
axes[1].set_ylabel("median of actual / predicted")
axes[1].set_title("2025: per-trip ratio by hour (1 = unbiased)")
fig.suptitle(f"Selected model: {best_name}", y=1.02)
```

```
plt.tight_layout()
plt.show()

print(f"Median actual {chosen['actual'].median():.2f} min | median predicted {chosen['predicted'].median():.2f} min")
print(f"Median of actual / predicted, all 2025 trips: {ratio.median():.3f}")
print(f"Hourly median ratio ranges from {ratio_by_hour.min():.3f} to {ratio_by_hour.max():.3f}")
```

Selected model: Linear, Huber



Median actual 11.52 min | median predicted 10.41 min

Median of actual / predicted, all 2025 trips: 1.006

Hourly median ratio ranges from 0.984 to 1.013

The chosen model holds up on a season it has never seen:

- **Linear, Huber: MAE 11.55, MedAE 1.91 min, 51.5% of trips within 2 minutes.** This is close to its 2024 validation results (1.86 min, 52.5%), so it works on a new season.
- **Compared with the constant baseline**, the typical error drops from 5.43 to 1.91 minutes (–65%), and trips within 2 minutes go from 18.6% to 51.5%.
- **The ranking is the same as in 2024.** Huber clearly beats squared error: 0.156 minutes of MAE and **8.2 percentage points within 2 minutes**, an even bigger gap than in 2024. It also beats the physics baseline by 0.40 minutes of MAE and 3.5 points.
- **Trees vs linear is still a tie.** Linear is 0.05 minutes better on MAE; the trees are 0.29 points better within 2 minutes (MedAE 1.890 vs 1.911). Both differences are statistically clear but too small to matter.
- **No bias.** Across all 2025 trips, the median of actual ÷ predicted is 1.006, and it stays between 0.98 and 1.01 for every hour (right panel). Comparing the *median actual* time (11.5 min) with the *median prediction* (10.4 min) would make the model look a minute too low, but that comparison is misleading. Actual times spread further above the prediction than below it, so when short and long routes are mixed, the median actual time ends up higher than the median prediction, even if every prediction is right. That's why we look at the ratio per trip instead.
- **MAE stays around 11.5 minutes while MedAE is about 1.9.** Forgotten rentals still dominate the *average* error, and no feature can predict them.

97.4% of 2025 trips had a known route length. 2.2% used their starting station's median, and 0.35% (trips from stations that opened in 2025) used the overall median.

14. Conclusions

Answer to the project question. Using the route, the time and the weather, the chosen model (linear regression on $\log(\text{duration})$ with the Huber loss) predicts 2025 trips with a typical error of **1.9 minutes**, and gets **half of all trips within 2 minutes**. A constant guess gets 5.4 minutes and 19%.

What adding the route changed. The Stage 1 features (time and weather) give about 1.5% of the possible improvement; route length gives 99.8%. Leaving out the route meant leaving out most of the answer. The trip's own covered distance is still not used: route length comes only from other, earlier trips on the same route.

The three assumptions:

- **Length:** confirmed, and the most important. Duration grows as $\text{length}^{0.85}$, so longer trips are ridden faster.
- **Time:** confirmed, but small. On the same route, speed changes by about 14 percentage points over the day, and weekends are 4–5% slower. This reflects why people ride (the morning commute is the fastest time of day), not traffic. It adds about 0.3 percentage points of trips within 2 minutes.
- **Weather:** not supported with daily data. Rain makes trips slightly *shorter and faster*, not slower, which corrects our Stage 1 report. Adding weather doesn't improve the predictions.

Loss function. The log transform and the Huber loss solve different parts of the problem. The log removes most of the instability caused by forgotten rentals. Huber removes the bias that remains: even on a log scale, squared error predicts normal trips about half a minute too long, and gets 6.4 (2024) and 8.2 (2025) percentage points fewer trips within 2 minutes than Huber. It is also about 12 times more sensitive to the longest trips.

Model type. The boosted trees don't add anything useful. They differ from the linear model by about 3 seconds of MAE or 0.3 points within 2 minutes, and which one wins depends on the metric. The near-linear idea from Stage 1 holds, so we keep the linear model: it was chosen by our pre-set rule, and it's simpler and easier to interpret.

Learning vs simple arithmetic. Dividing route length by one typical speed (11.5 km/h) already gets 61 of the 65 percentage points of improvement in typical error. Learning adds a real but small improvement, about 3.5 percentage points more trips within 2 minutes. Most of what makes a trip's duration predictable is simply how far it is.

Limitations:

- **Daily weather is rough.** A rainy day might have been dry at 17:00. Hourly data is the last chance for weather to show an effect on speed.
- **Route lengths come from history**, so 2.6% of 2025 trips (new stations and rare routes) use a rougher estimate.

- **Rider behaviour can't be predicted.** Stops, detours and forgotten rentals keep the *average* error around 11.5 minutes, even though the *typical* error is under 2.
- **All models are trained on a 400,000-trip sample.** We didn't test whether more data helps.
- **The bootstrap intervals only cover variation within one season,** not between seasons; the 2025 test is our only check on that.
- **Round trips (3.2%) behave differently** and are only handled with a flag.